Notebook setup

Run the next cell first. In Google Colab it clones the course repository and installs the extra packages, then imports the libraries. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Bayesian optimization using Gaussian processes

If every function evaluation is expensive, for instance when each evaluation is a bioreactor experiment that takes two weeks, optimizing the process parameters with standard optimization routines (which need hundreds of evaluations) would take forever.

The idea is to approximate the function with a Gaussian process. In other words, the function values are assumed to follow a multivariate Gaussian distribution whose covariance is given by a GP kernel between the parameters. A smart choice of the next parameter to evaluate is then made with an acquisition function computed on the GP posterior, which is much quicker to evaluate than the real function.

Problem definition

We are interested in solving

$$x^* = \arg\max_x f(x) \tag{1}$$

under the constraints that:

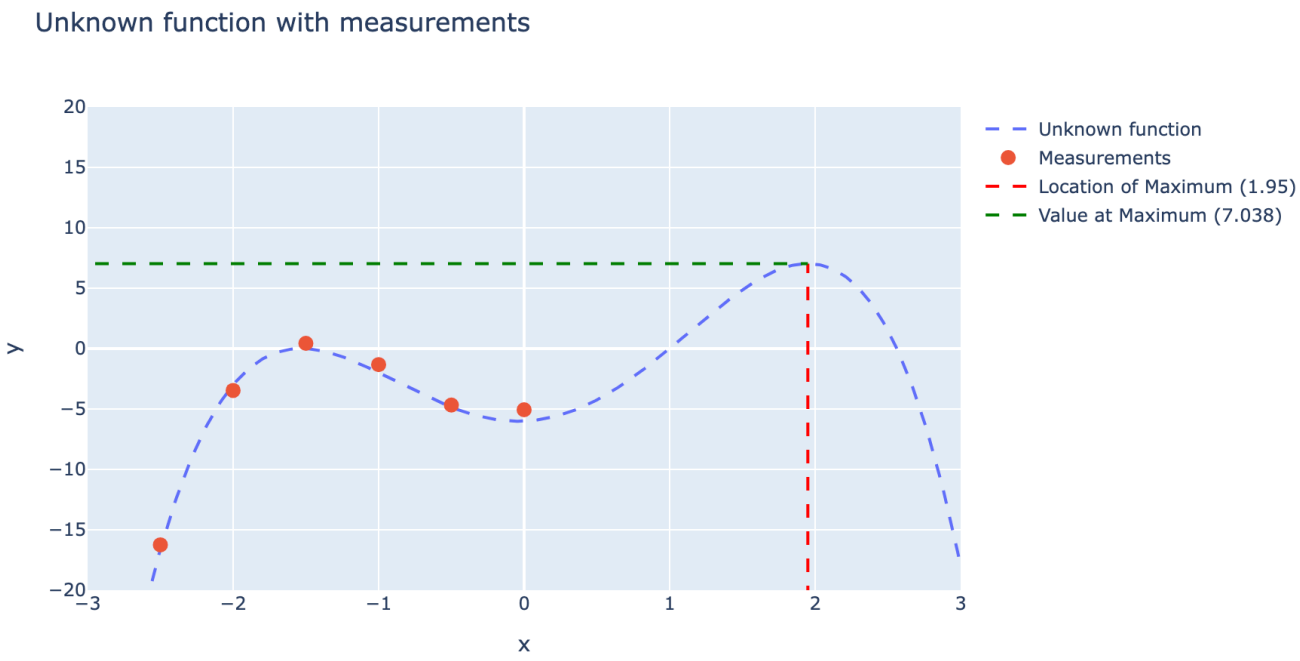
- f is a black box for which no closed form is known (nor its gradients)
- f is expensive to evaluate
- evaluations of $y = f(x)$ may be noisy

Create the function to optimize

Here we first create an originator function that is nonlinear, and we add noise to its observations. A few initial measurements are taken on the left side of the domain.

```
""" Fix seed to control randomness """
np.random.seed(42)
""" Noise in function observations """
EPS = 0.5
""" Define initial experimental points locations """
XPOINTS = np.arange(-2.5,0.5,0.5)
```

```
# Function definition (unknown to the optimizer)
xgrid = np.round(np.arange(-3,3,0.01),4)
def f(x,eps=0):
    return
-1*np.power(x,4)+0.5*np.power(x,3)+6*np.power(x,2)+0.5*np.power(x,1)-6+eps*np.random.
normal(0,1,x.size)
fgrid = f(xgrid)
# Obtain observation measurements with noise
xpoints = XPOINTS
ypoints = f(xpoints,EPS)
```

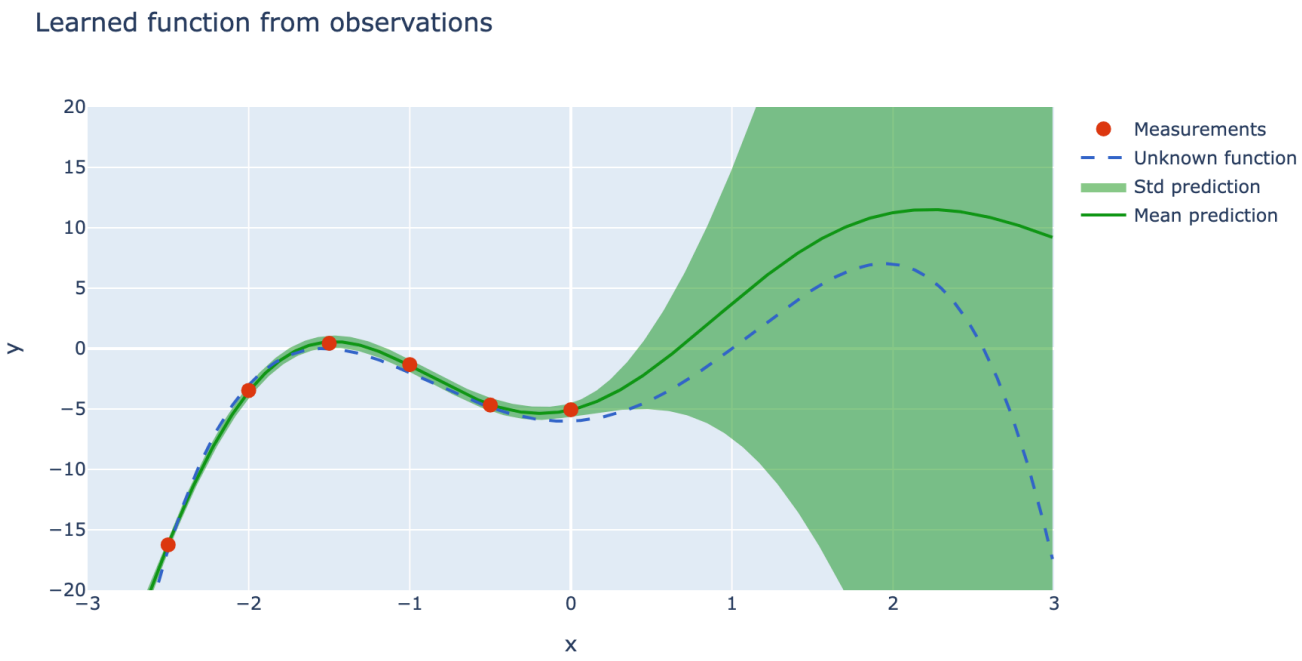


The goal is to find the maximum. If we could sample our function an unlimited number of times, we could just look up the answer. However, we have limited resources, so we can only take a few experimental observations and have to approximate (model) the behaviour in the remaining places.

Fit a Gaussian process model

We use a Gaussian process with a standard RBF kernel to learn the function above from the measurements.

```
# Define Model
kernel = 1.0 * RBF(length_scale=1e-1, length_scale_bounds=(1e-3, 1e3)) +
WhiteKernel(noise_level=1e-2, noise_level_bounds=(1e-2, 1e1))
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(xpoints.reshape(-1,1), ypoints.reshape(-1,1))
# Use model to make predictions
xpred = xgrid
mean, std = gpr.predict(xpred.reshape(-1, 1), return_std=True)
mean, std = mean.ravel(), std.ravel()
ypred = np.vstack([mean - 2*std, mean, mean + 2*std])
```



Define the acquisition function

The acquisition function represents our objective for choosing the next experiment. It guides us towards the maximum value and its location.

Exploration / exploitation

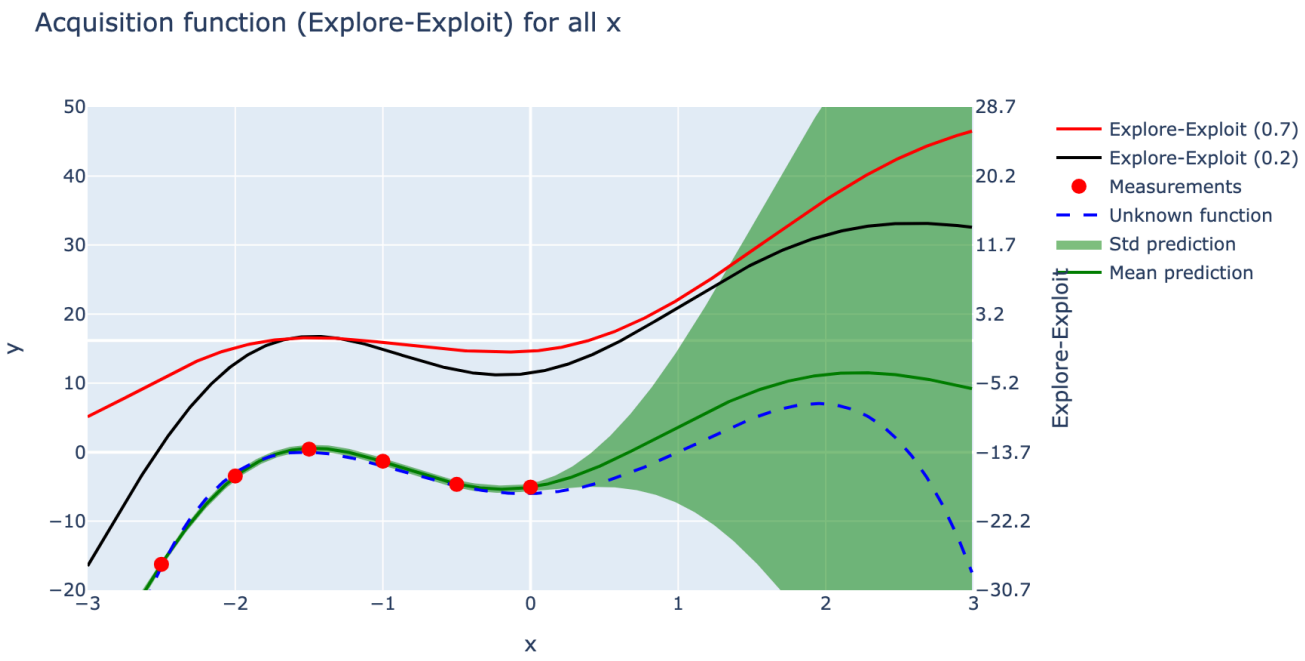
The simplest acquisition function one can think of is the conditional mean itself. This fully exploits the current model. We can add the uncertainty estimate as an exploration term to also investigate regions with high uncertainty or few observations.

You can define your own acquisition function. The function demonstrated below balances exploration and exploitation with the parameter κ = `KAPPA`. It is a weighted version of the upper confidence bound (UCB):

$$UCB(x) = (1 - \kappa)\,\mu_{GP}(x) + \kappa\,\sigma_{GP}(x) \tag{2}$$

```
""" Exploration / Exploitation tradeoff """
KAPPA1 = 0.2
KAPPA2 = 0.7
```

```
# Calculate EE (Explore-Exploit)
eegrid1 = (1-KAPPA1)*ypred[1,:] + KAPPA1*((ypred[1,:]-ypred[0,:])/2)
eegrid2 = (1-KAPPA2)*ypred[1,:] + KAPPA2*((ypred[1,:]-ypred[0,:])/2)
```



Probability of improvement

Another good acquisition function is the probability of improvement: the probability that the function value at x exceeds the best value observed so far, $f(x^+)$,

$$PI(x) = P\left(f(x) \geq f(x^+)\right) = 1 - \Phi\left(\frac{f(x^+) - \mu_{GP}(x)}{\sigma_{GP}(x)}\right) \tag{3}$$

where Φ is the cumulative distribution function of the standard normal distribution. To demonstrate it, we evaluate the model at a new point and compare the prediction with the maximal value observed so far.

```
""" New evaluation point """
XNEW = 1.5
```

```
# Obtain prediction at the new point
xnew = XNEW
(xnew_mean,xnew_std) = gpr.predict(np.array(xnew).reshape(1,-1),return_std=True)
xnew_mean, xnew_std = xnew_mean.item(), xnew_std.item()
print('Predicted mean and std at x = '+str(xnew))
print("Mean: ",float(xnew_mean))
print("Std: ",float(xnew_std))

# Actual maximum value
y_max = np.max(ypoints)
print("Observed maximum function value so far at y = "+str(round(y_max,4)))
```

```
Predicted mean and std at x = 1.5
Mean:  8.662260233230706
Std:  12.043285943567412
Observed maximum function value so far at y = 0.4348
```

Compute the distribution

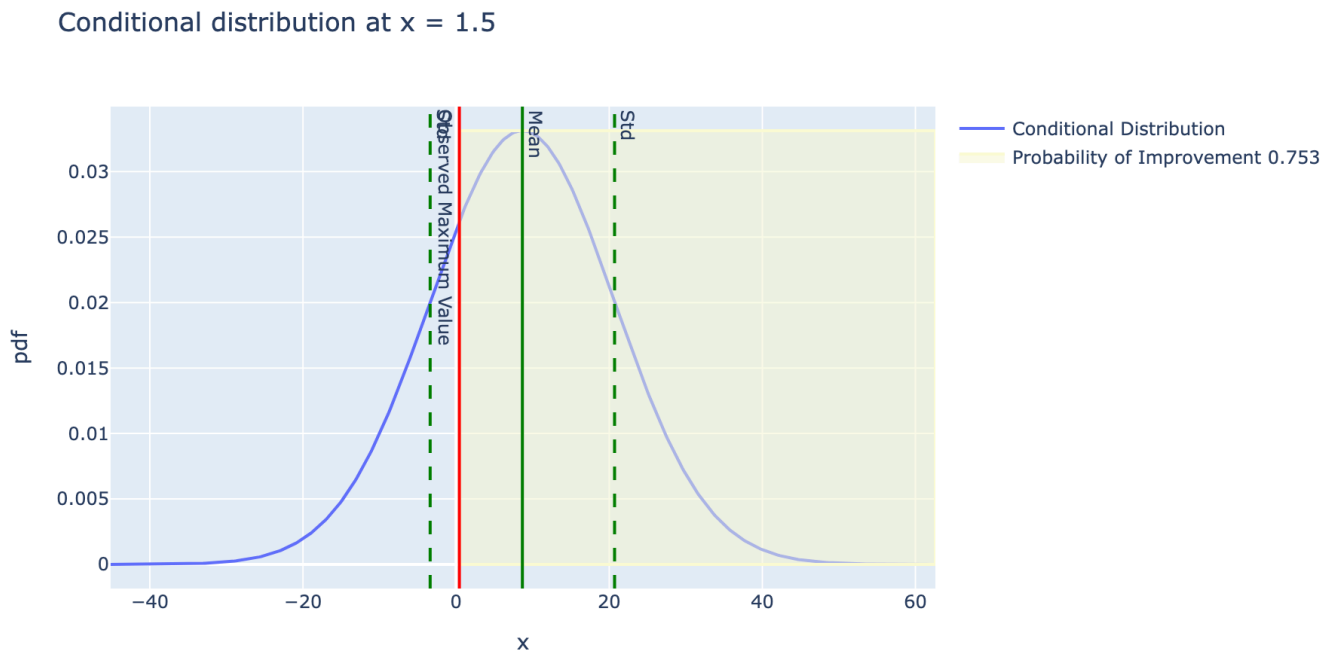
Since the prediction at the new point `XNEW` is normally distributed with mean `xnew_mean` and standard deviation `xnew_std`, we can plot the full distribution at the new point.

This distribution represents the possible observations according to the model. The solid red line is the actually observed maximum value so far, `y_max`.

The probability of improvement is the probability of observing a value from this conditional distribution that is greater than the observed maximum: the shaded area to the right of the red line.

```
# Probability of improvement at the new point
PI = 1-norm(float(xnew_mean), float(xnew_std)).cdf(y_max)
print("Probability of improvement at x = "+str(xnew)+": ", round(PI,4))
# Samples from the conditional distribution (for the plot range)
xdist = np.random.randn(100000)*float(xnew_std) + float(xnew_mean)
hist_data = [xdist]
group_labels = ['Conditional Distribution']
```

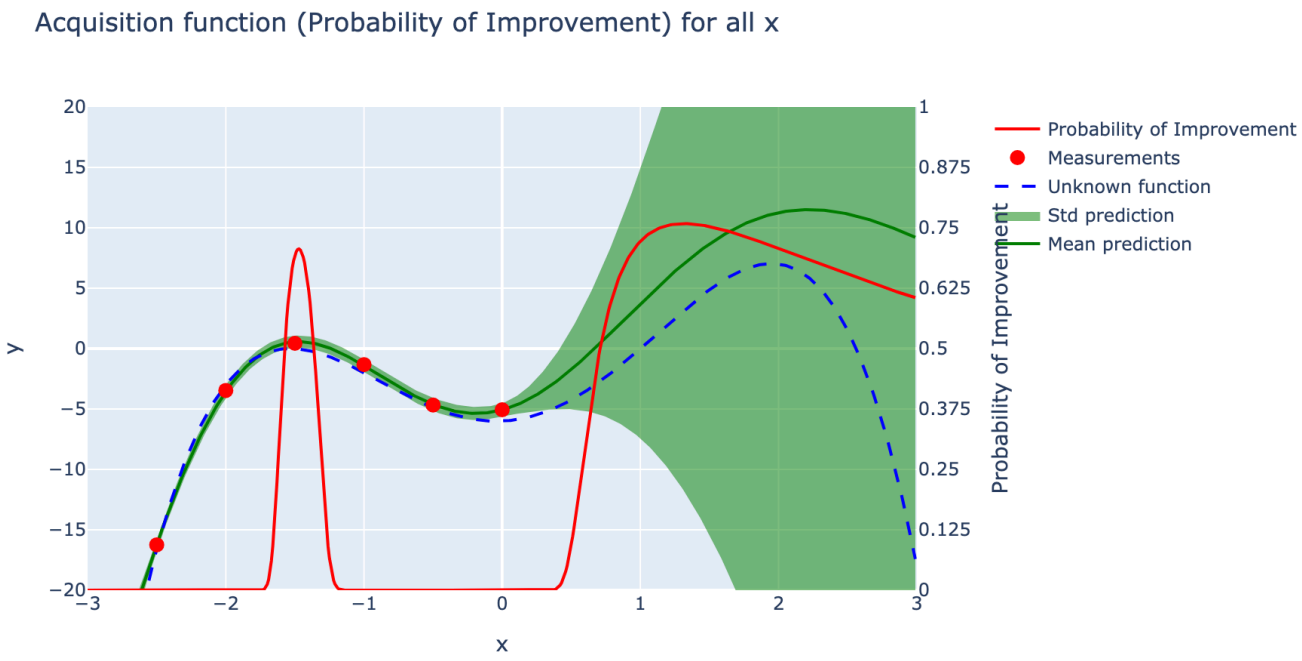
```
Probability of improvement at x = 1.5:  0.7527
```



Compute the acquisition function for all x

Now we compute the probability of improvement for all x values of the grid.

```
# Calculate PI (Probability of Improvement)
pigrid = np.zeros_like(xgrid)
for n in range(len(xgrid)):
    pigrid[n] = 1-norm(float(ypred[1,n]), float((ypred[1,n]-
ypred[0,n])/2)).cdf(y_max)
```



Given the acquisition function, the next point at which we would like to take a measurement is the value of x for which the acquisition function is the highest.

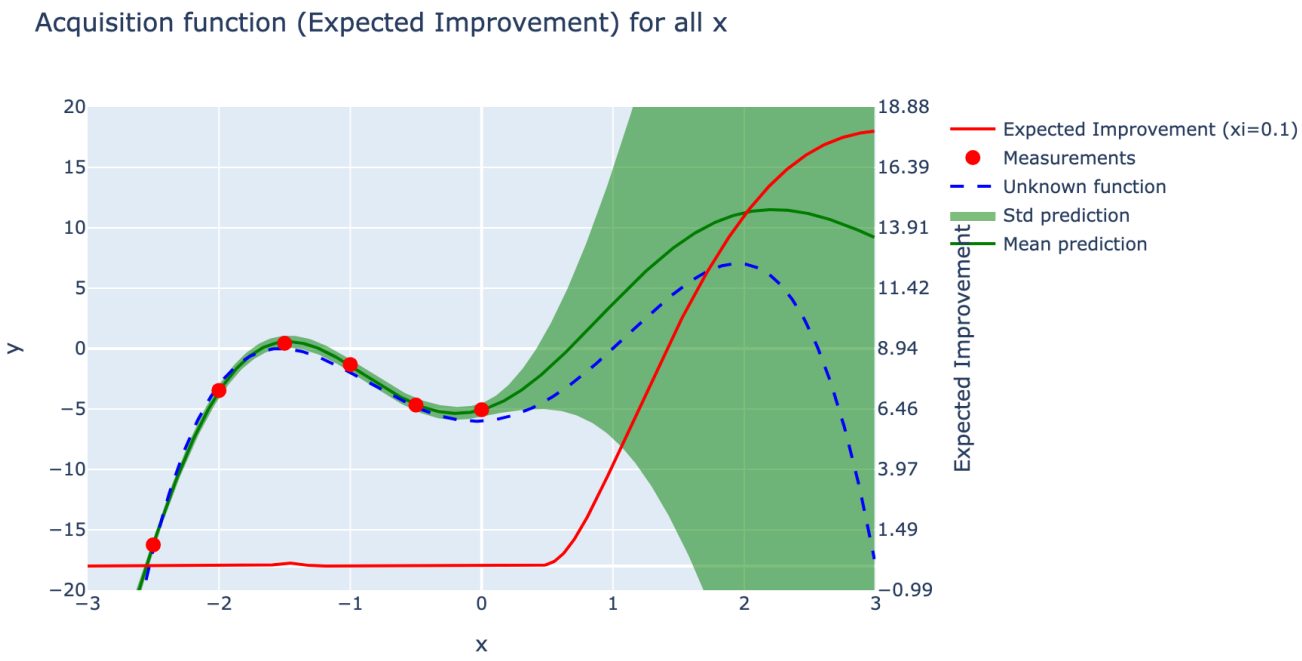
Expected improvement

Here we modify the acquisition function to the expected improvement. It weights the amount by which we expect to improve on the best observation with the probability of this improvement:

$$EI(x) = E\left[\max\left(f(x) - f(x^+) - \xi,\, 0\right)\right] \tag{4}$$

The parameter ξ = `xi` controls the exploration: larger values favour points with high uncertainty.

```
# Calculate EI (Expected Improvement) with the closed-form expression for a Gaussian posterior
eigrid = np.zeros_like(xgrid)
xi = 0.1
for n in range(len(xgrid)):
    sigma = float((ypred[1,n]-ypred[0,n])/2)
    zscore = (float(ypred[1,n])-y_max-xi) / sigma
    C = norm(0,1).cdf(zscore)
    P = norm(0,1).pdf(zscore)
    eigrid[n] = (float(ypred[1,n])-y_max-xi)*C + sigma*P
```



Run Bayesian optimization

Now that we have defined the optimization problem, we run the Bayesian optimization with the `bayesian-optimization` package and see whether we are able to iteratively find the maximum.

Target function

Let's define a 1-D target function with multiple local maxima to test and visualize how the `BayesianOptimization` package works. Two target functions are prepared (the second one is commented out):

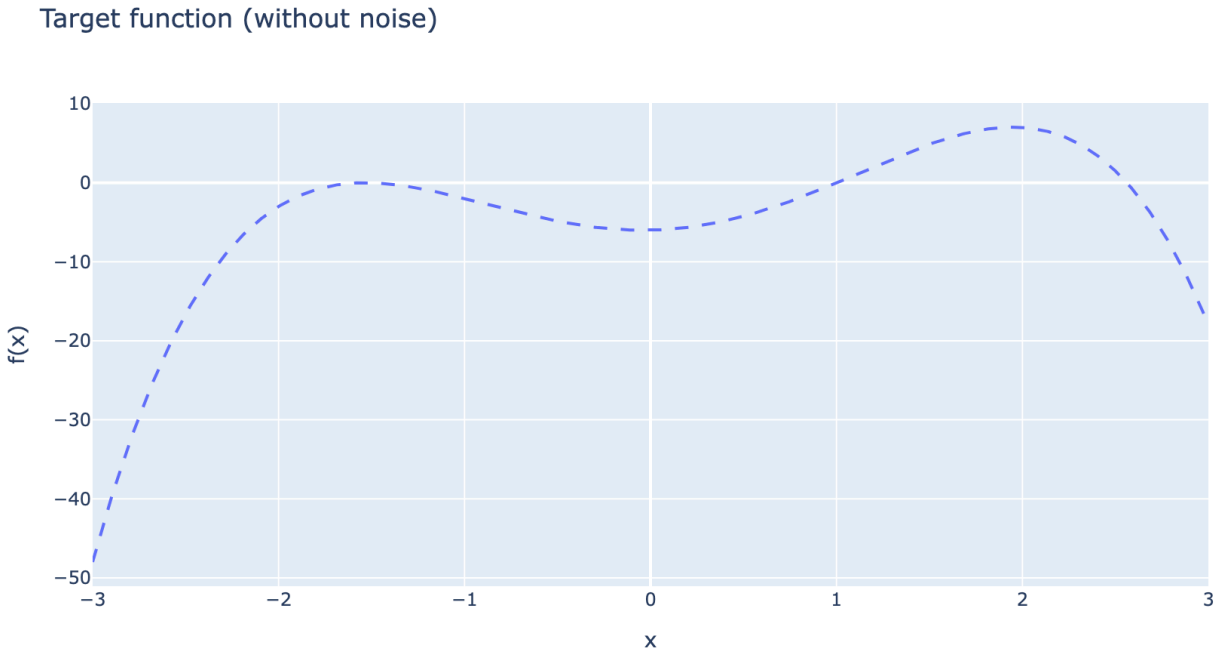
$$f_1(x) = -x^4 + 0.5x^3 + 6x^2 + 0.5x - 6 + \epsilon \tag{5}$$

$$f_2(x) = e^{-(x-2)^2} + e^{-0.1(x-6)^2} + 1/(x^2+1) \tag{6}$$

The maximum of both is around $x = 2$ and we restrict the interval of interest to $x_{f_1} \in (-3, 3)$ and $x_{f_2} \in (-2, 10)$.

Notice that in practice this function is unknown; the only information we have is obtained by sequentially probing it at different points. Bayesian optimization works by constructing a posterior distribution of functions that best fits the observed data and choosing the next probing point by balancing exploration and exploitation.

```
""" Noise level """
EPS = 0.5
""" Function to optimize """
def f(x,eps=0.5):
    x = np.array(x)
    return -1*(x**4)+0.5*(x**3)+6*(x**2)+0.5*x-6 + eps*np.random.normal(0,1,x.shape)
# domain (-3,3)
    #return np.exp(-(x - 2)**2) + np.exp(-(x - 6)**2/10) + 1/ (x**2 + 1)   # domain
(-2,10)
    #return np.sin(x)/2 - ((10 - x)**2)/50 + 2 # use domain (0,20)
""" Domain / range of values"""
domain = (-3,3)
#domain = (-2,10)
#domain = (0,20)
```



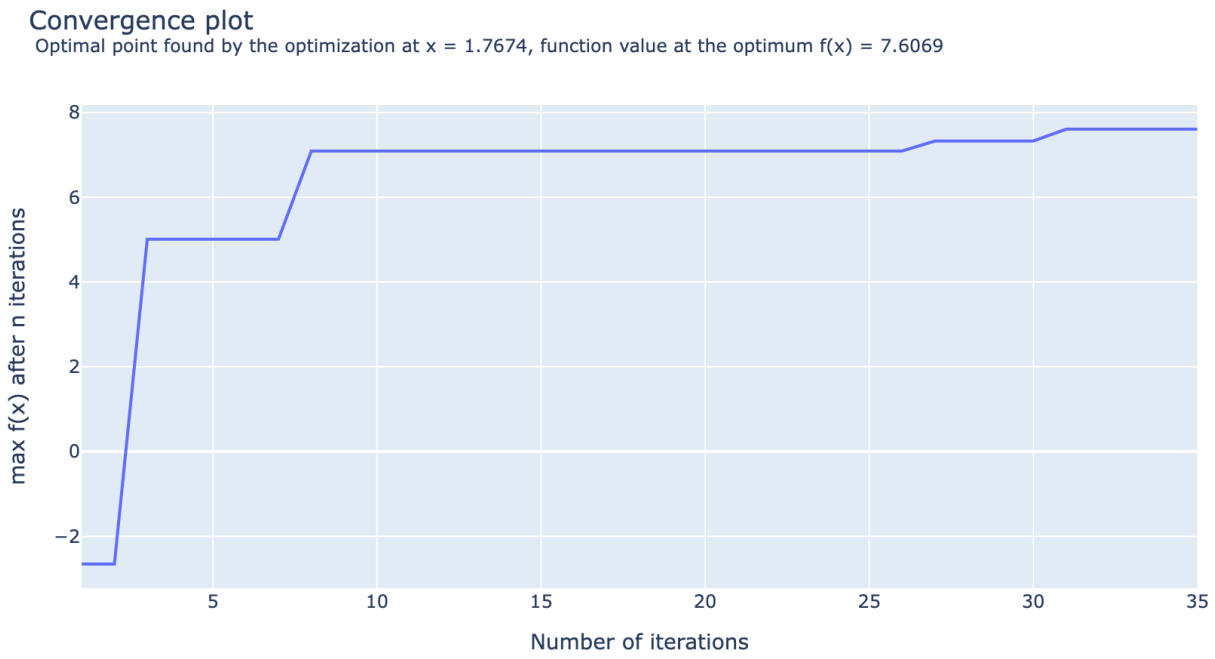
We pass the target function to be maximized, its variable and the corresponding range to the optimizer. A minimum of 2 initial guesses is necessary to kick-start the algorithm; these are chosen at random here. The optimizer then runs 30 iterations, each time fitting the GP and probing the point with the highest acquisition value.

```
# Define optimizer (verbose=2 prints every probed point)
optimizer = BayesianOptimization(f, {'x': domain}, random_state=42, verbose=2)
# alpha is the assumed noise variance of the observations
optimizer._gp.set_params(**dict(alpha=1))
# Run the optimization: 5 random initial points followed by 30 Bayesian iterations
# No acquisition_function is passed, so bayes_opt uses its default: UCB with
kappa=2.576
optimizer.maximize(init_points=5, n_iter=30)
```

iter	target	x
1	-2.653	-0.7528
2	-4.248	2.704
3	5.012	1.392
4	-3.705	0.592
5	-4.071	-2.064
6	3.789	1.365
7	4.396	1.594
8	7.091	1.911
9	-48.5	-3.0
10	-17.64	3.0
11	-6.298	-0.0273
12	-0.9223	0.9395
13	-1.294	-1.11
14	6.698	1.948
15	6.672	1.94
16	7.001	1.919
17	4.571	1.386
18	6.66	2.085
19	0.8745	-1.369
20	6.649	1.658
21	6.335	2.134
22	5.685	1.579
23	-5.043	-0.4214
24	6.676	2.091
25	6.065	1.638
26	1.112	1.158
27	7.326	2.099
28	6.796	1.8
29	6.839	2.159
30	6.314	1.753
31	7.607	1.767
32	6.067	2.182
33	6.059	1.73
34	5.632	1.777
35	6.256	2.183

Convergence plot

The best function value found so far is plotted against the number of iterations.



Acquisition function progression

To understand what happens in every iteration, we now run the optimization step by step and plot the GP posterior (top) together with the acquisition function (bottom). The same acquisition function `acq_fn` (upper confidence bound with `kappa=KAPPA`) is used both for the plot and by the optimizer to choose the next point, so the star marks the point that is probed in the next step.

Define the optimization that we investigate sequentially. It starts with 2 random points and uses a Matern kernel in the GP.

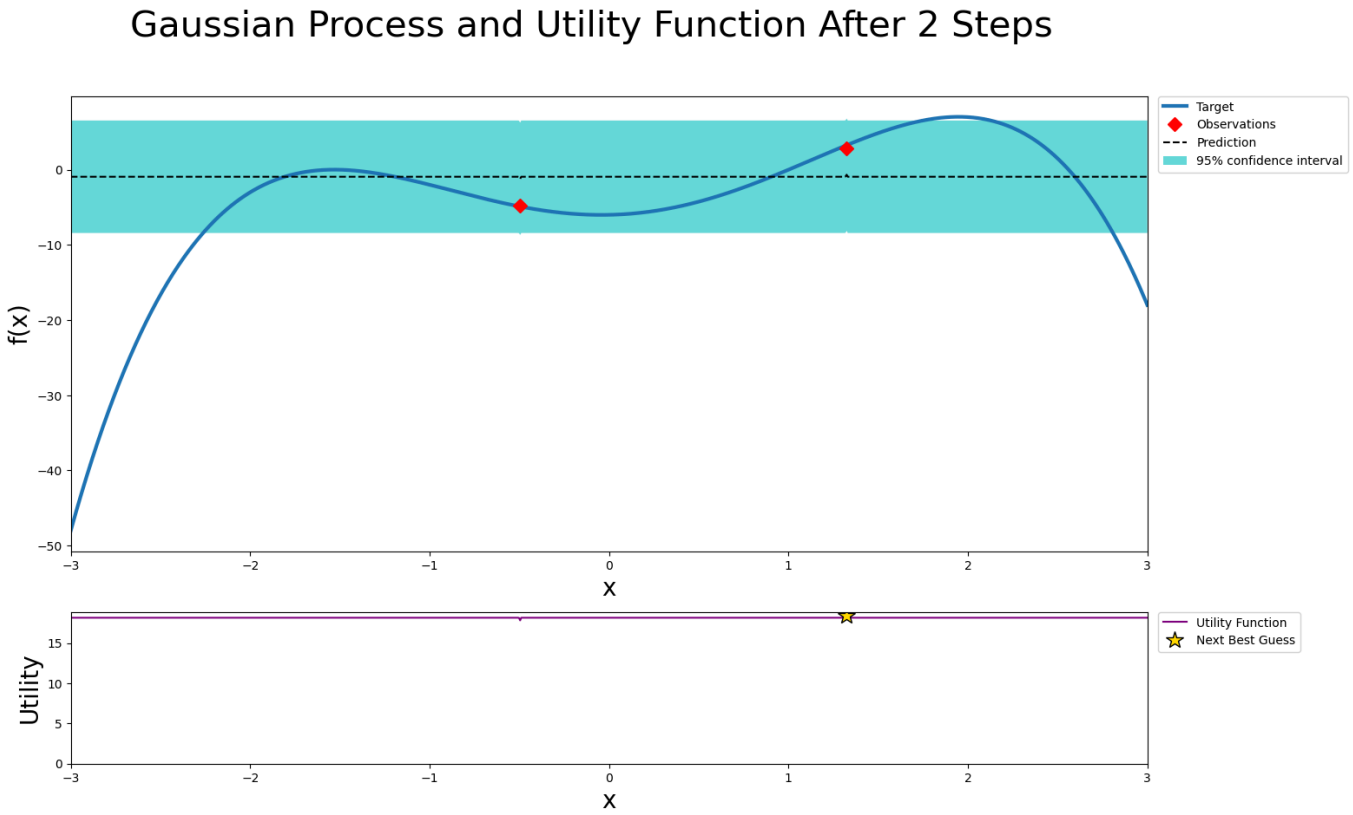
```
# Grid for plotting
x = xgrid.reshape(-1, 1)
y = f(xgrid,eps=0)
# Define the optimization and probe 2 random initial points
optimizer = BayesianOptimization(f, {'x': domain}, random_state=1, verbose=2)
optimizer.maximize(init_points=2, n_iter=0)
# Modify the GP model: Matern kernel and observation noise
kernel = Matern(length_scale=1, nu=2.5, length_scale_bounds=(1e-3,1e3))
optimizer._gp.set_params(**dict(alpha=0.1, kernel=kernel))
print("GP kernel:", optimizer._gp.kernel)
# Acquisition function used both for plotting and for choosing the next point
KAPPA = 5
acq_fn = UtilityFunction(kind="ucb", kappa=KAPPA, xi=0)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 1         | -4.751    | -0.4979   |
| 2         | 2.882     | 1.322     |
=====================================
GP kernel: Matern(length_scale=1, nu=2.5)
```

Two random points

After probing two points at random, we can fit a Gaussian process and start the Bayesian optimization procedure. Two points give an uneventful posterior with the uncertainty growing as we move away from the observations.

```
plot_gp(optimizer, x, y)
```

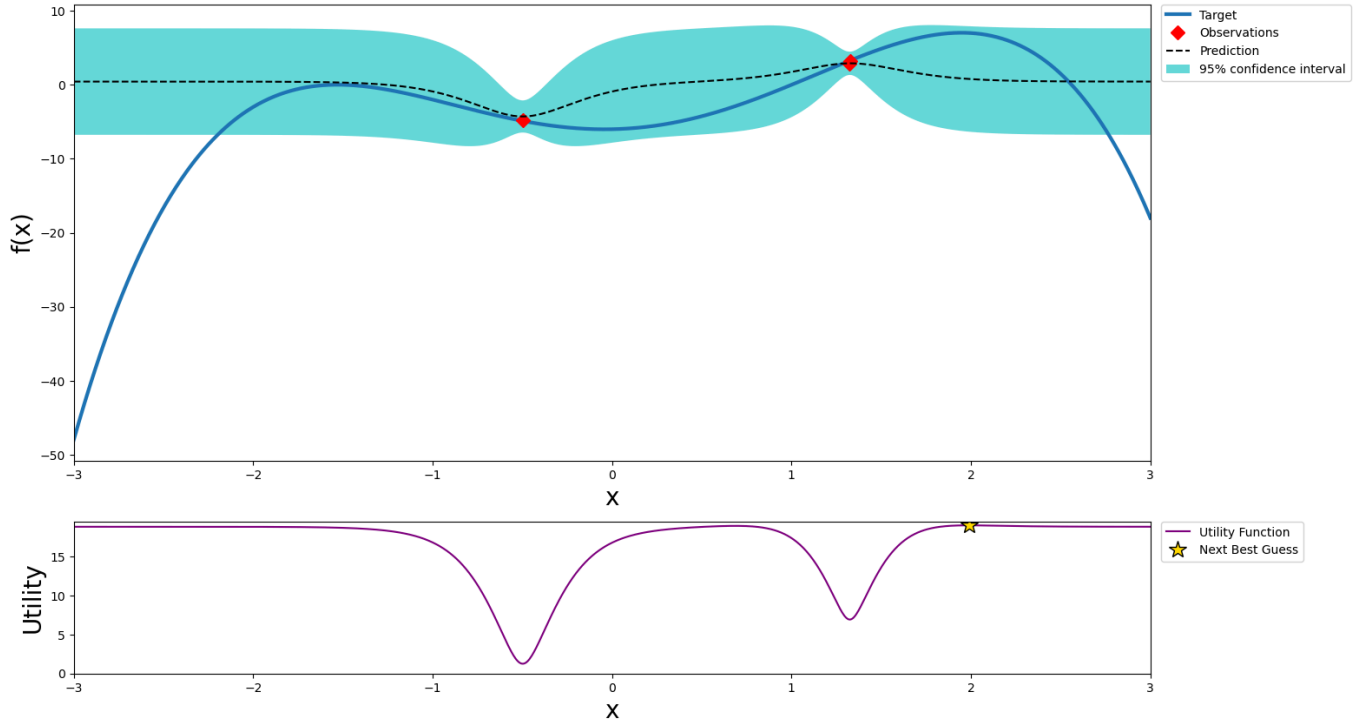


After one step of Bayesian optimization

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 3         | 3.179     | 1.328     |
=====================================
```

Gaussian Process and Utility Function After 3 Steps

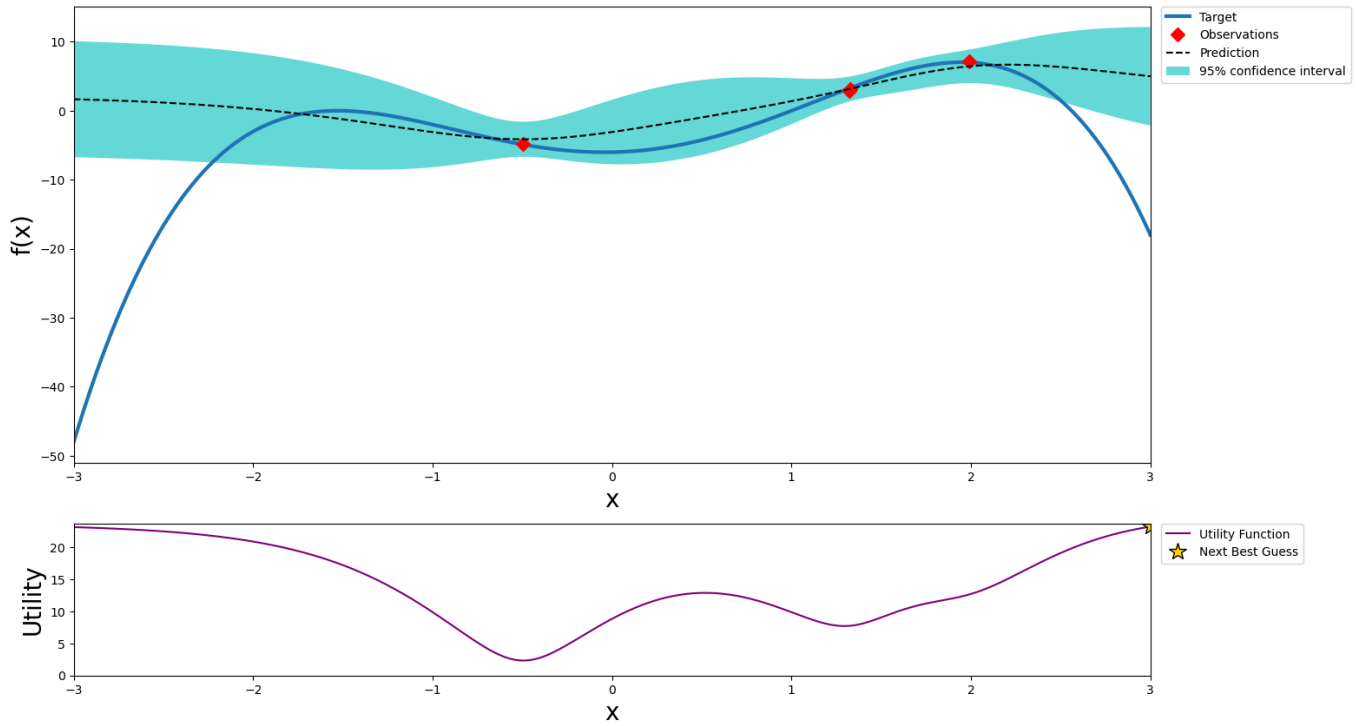


After two steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 4         | 7.119     | 1.989     |
=====================================
```

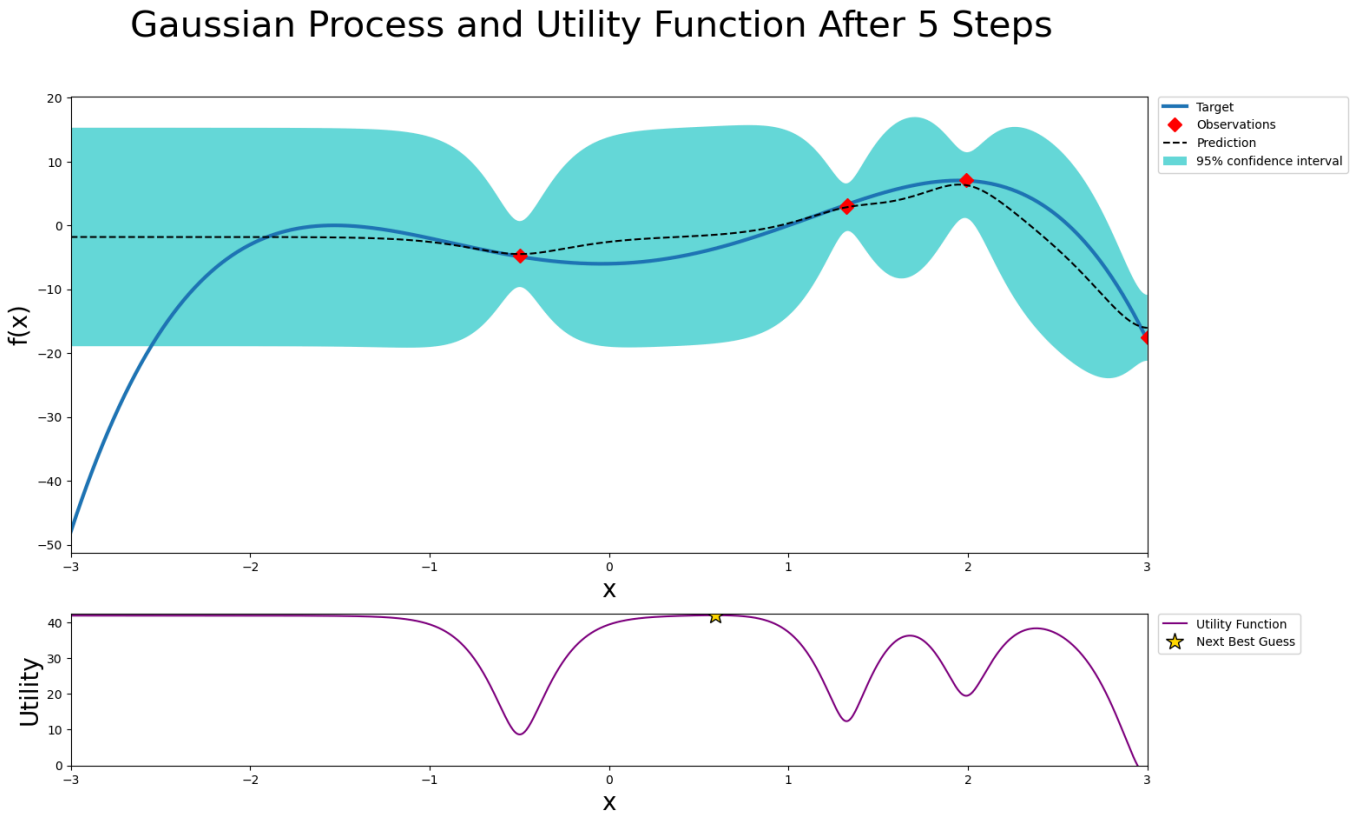
Gaussian Process and Utility Function After 4 Steps



After three steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

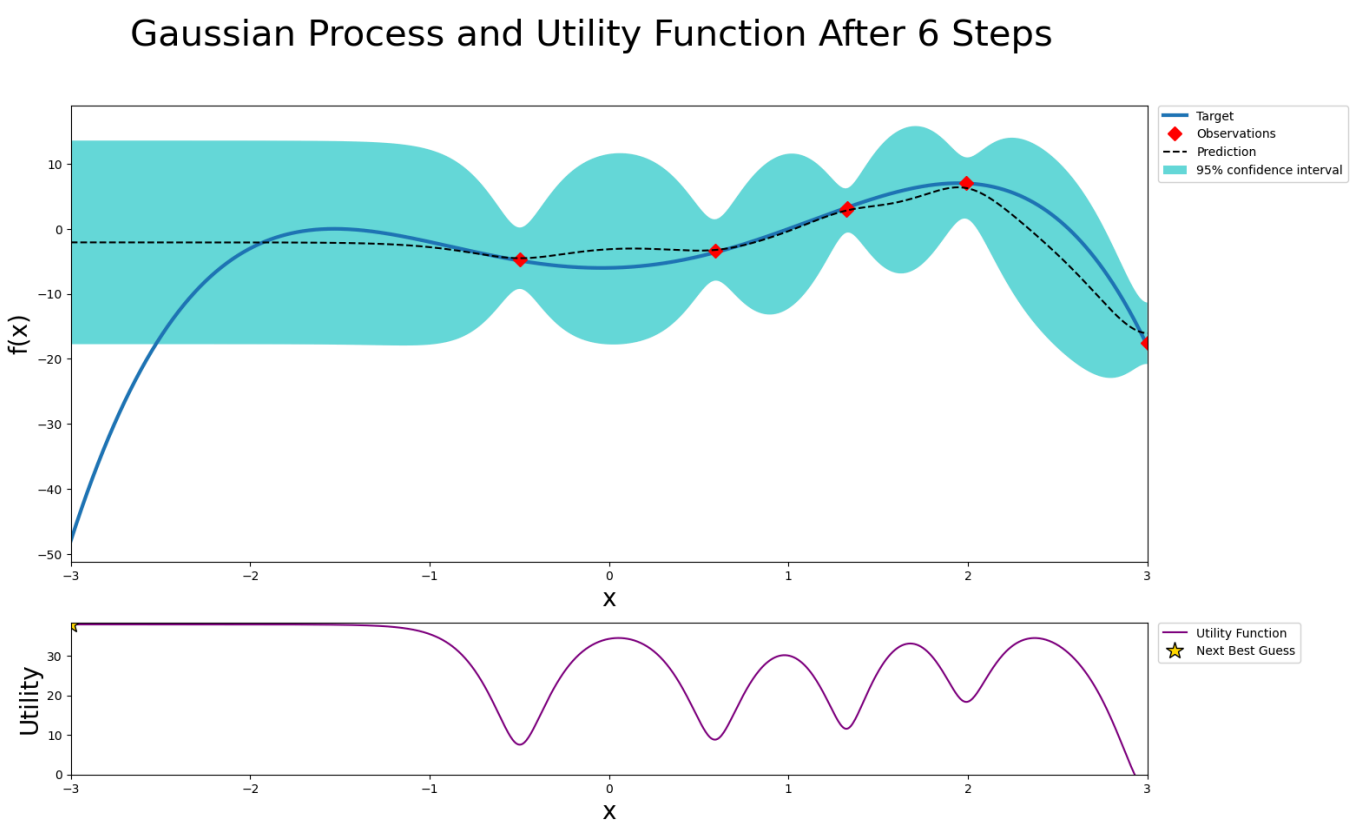
```
|   iter    |  target   |     x     |
-------------------------------------
| 5         | -17.48    | 3.0       |
=====================================
```



After four steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

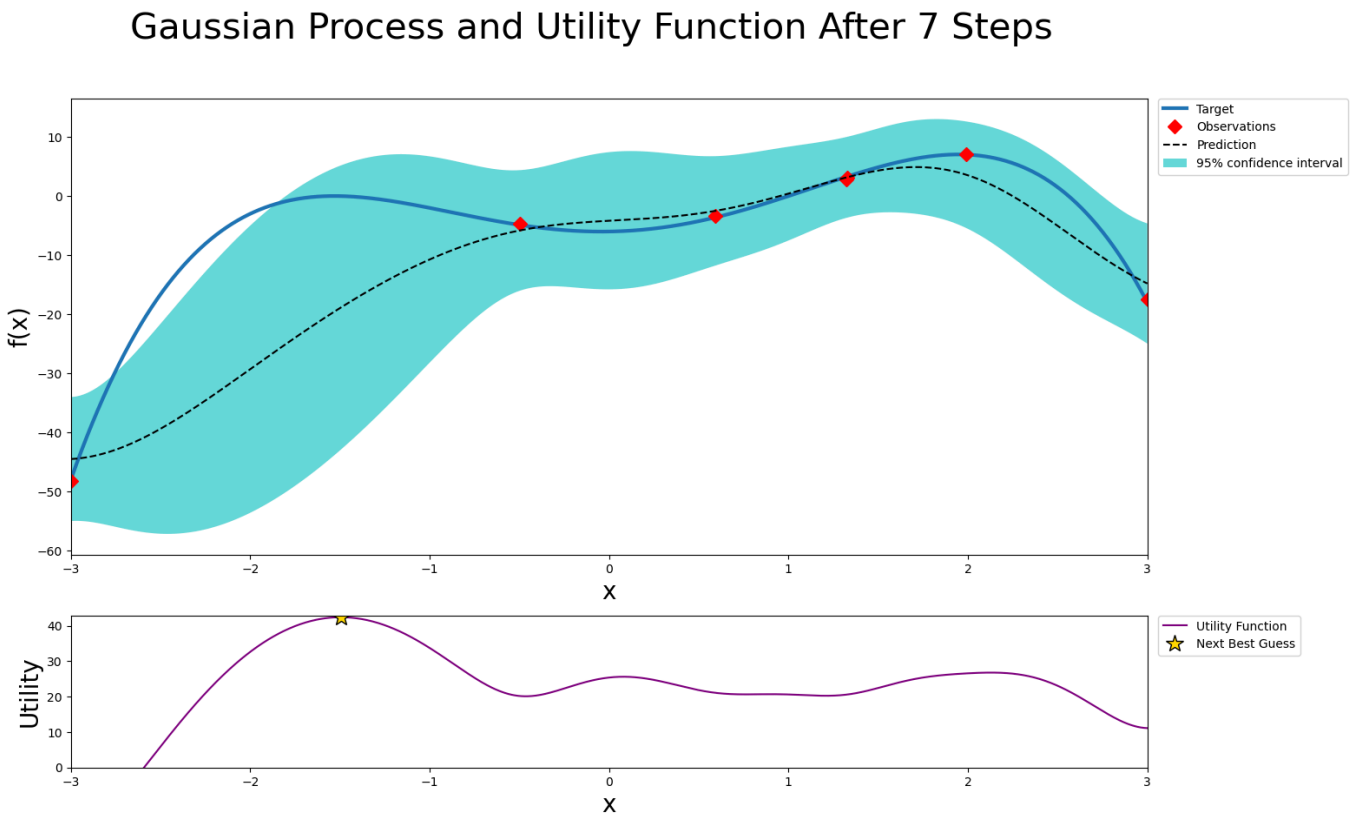
```
|   iter    |  target   |     x     |
-------------------------------------
| 6         | -3.399    | 0.5909    |
=====================================
```



After five steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 7         | -48.2     | -3.0      |
=====================================
```

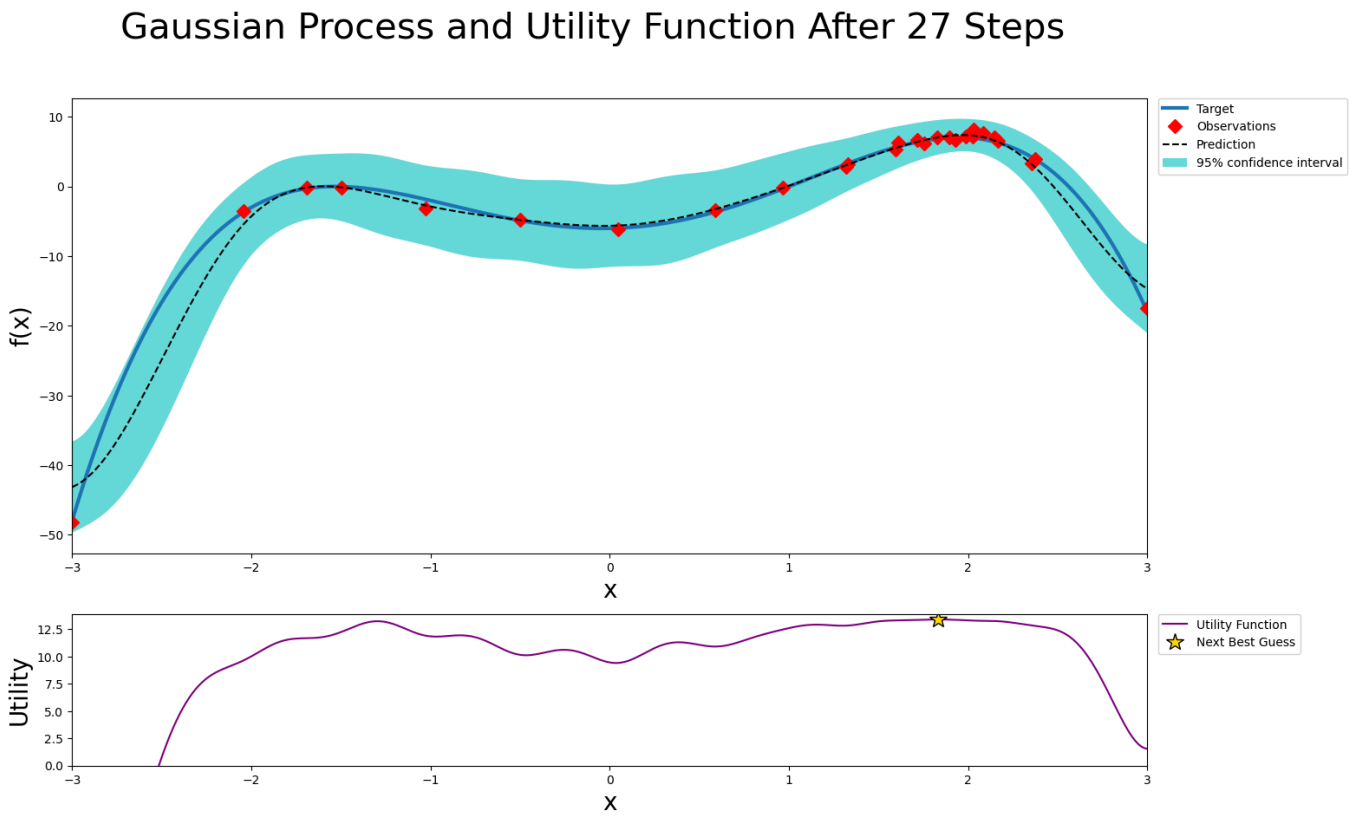


After 25 steps

After 20 more iterations the GP describes the function well around the maximum, while regions far from the optimum remain uncertain: the optimizer does not waste experiments there.

```
optimizer.maximize(init_points=0, n_iter=20, acquisition_function=acq_fn)
plot_gp(optimizer, x, y)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 8         | -0.1635   | -1.495    |
| 9         | -3.568    | -2.041    |
| 10        | -6.079    | 0.04734   |
| 11        | 3.936     | 2.373     |
| 12        | -3.17     | -1.026    |
| 13        | 6.148     | 1.757     |
| 14        | 7.63      | 2.084     |
| 15        | -0.1335   | 0.9645    |
| 16        | 6.64      | 1.716     |
| 17        | 6.513     | 2.168     |
| 18        | 7.023     | 1.825     |
| 19        | -0.2007   | -1.689    |
| 20        | 6.319     | 1.61      |
| 21        | 7.209     | 2.025     |
| 22        | 3.323     | 2.357     |
| 23        | 6.686     | 1.931     |
| 24        | 5.307     | 1.593     |
| 25        | 8.127     | 2.029     |
| 26        | 7.059     | 1.894     |
| 27        | 7.107     | 2.148     |
=====================================
```



Tasks

1. Switch to the second target function f_2 (uncomment it in the function definition and set `domain = (-2,10)`). Does the optimizer find the global maximum or does it get stuck in a local one?
2. Change `KAPPA` (defined where the step-by-step optimizer is created) to 1 (exploitation) and to 10 (exploration) and repeat the step-by-step optimization. How does the sequence of probed points change?
3. Increase the noise `EPS` to 2. How many more iterations does the optimizer need to find the maximum?

```
# Your code here
```